



Dolarito — Cotizaciones & Mercado en Tiempo Real

Entregable 02 · Consultas (Querys) ABM de Usuarios · v1.0 · Octubre 2026

Consultas (Querys) · ABM de usuarios

Documento técnico con todas las sentencias SQL (PostgreSQL) que implementan el **Alta, Baja y Modificación** de usuarios —y las consultas de soporte— en el micro-emprendimiento **Dolarito (divise.)**. La base de datos corre sobre **PostgreSQL** en **Supabase**; cada sentencia se indica también con su ruta de API y la operación del frontend que la dispara.

1. Esquema de la tabla `usuarios`

La tabla central del ABM fue evolucionando con migraciones (001 → 010). Columnas vigentes:

Columna	Tipo	Descripción
<code>id_usuario</code>	INTEGER · PK (SERIAL)	Identificador único del usuario
<code>auth_user_id</code>	UUID (NULL)	Referencia al auth de Supabase (login social, opcional)
<code>nombre</code>	VARCHAR(100)	Nombre y apellido
<code>email</code>	VARCHAR(150) · UNIQUE	Correo electrónico (clave de acceso)
<code>password_hash</code>	VARCHAR(255)	Hash bcrypt de la contraseña (nunca clave en texto plano)
<code>divisa_base_id</code>	INTEGER · FK → divisas	Moneda base del usuario (configurable en perfil)
<code>email_verificado</code>	BOOLEAN	¿Confirmó su email con el código de 6 dígitos?
<code>two_factor_enabled</code>	BOOLEAN	2FA: se envía código por correo al iniciar sesión
<code>verif_codigo / verif_expira / verif_token</code>	VARCHAR · TIMESTAMP · VARCHAR	Código de verificación de email y su enlace
<code>reset_token / reset_token_expires</code>	VARCHAR · TIMESTAMP	Código de recuperación de contraseña (30 min)
<code>whatsapp_phone / whatsapp_api_key</code>	VARCHAR	Telegram/CallMeBot para notificaciones (alertas)

<code>activo</code>	BOOLEAN	Estado del registro (baja lógica, default TRUE)
<code>deleted_at</code>	TIMESTAMP (NULL)	Marca de baja lógica
<code>created_at / updated_at</code>	TIMESTAMP	Auditoría de creación y última modificación

Integridad referencial: las tablas `favoritos`, `historial_de_consultas` y `alertas` referencian a `usuarios` con `ON DELETE CASCADE`.

2. INDEX del ABM por operación

Operación	Endpoints (API REST)	Pantalla
A Alta	<code>POST /api/auth/register</code> · seed de prueba	Registro / Crear cuenta
C Consulta	<code>POST /api/auth/login</code> · <code>GET /api/users/me</code> · <code>GET /api/users</code> · <code>GET /api/users/:id</code>	Login · Perfil · (listado)
M Modificación	<code>PUT /api/users/me</code> · <code>PUT /api/users/me/password</code> · <code>PUT /api/users/me/2fa</code> · <code>PUT /api/users/me/whatsapp</code>	Editá tu perfil
B Baja	<code>DELETE /api/users/me</code> (y <code>DELETE /api/users/:id</code>)	Eliminar cuenta

3. ALTA — Crear usuario (Create)

3.1 Registro de un nuevo usuario **A**

Objetivo: dar de alta una cuenta nueva. La contraseña se hashea antes con bcrypt (12 rondas) en el backend; la sentencia `RETURNING` devuelve los datos mínimos para emitir el token de sesión.

Entrada: `nombre`, `email`, `password_hash` · **Ruta:** `POST /api/auth/register` (función `createUser` en `userModel.js`)

```
INSERT INTO usuarios (nombre, email, password_hash, email_verificado, activo)
VALUES ('María García', 'maria@example.com', '$2b$12$3f...hashBcrypt...', FALSE, TRUE)
RETURNING id_usuario, nombre, email, divisa_base_id;
```

3.2 Usuario de prueba (semilla) **A**

Objetivo: registrar la cuenta demo entregada al profesor (consulta utilizada también en `seed.sql`).

```
INSERT INTO usuarios (nombre, email, password_hash)
VALUES ('Demo Divise', 'demo.divise@example.com', '$2b$12$...hashDeDemo1234!...')
RETURNING id_usuario, nombre, email;
```

4. CONSULTA — Leer usuarios (Read)

4.1 Consulta de login por email C

Objetivo: localizar la cuenta al iniciar sesión y validar el hash.

Ruta: `POST /api/auth/login` (`getUserByEmail`)

```
SELECT * FROM usuarios
WHERE email = 'maria@example.com'
LIMIT 1;
```

4.2 Perfil del usuario autenticado C

Objetivo: devolver los datos visibles del perfil junto con la moneda base.

Ruta: `GET /api/users/me` (id desde el token JWT)

```
SELECT u.id_usuario, u.nombre, u.email, u.divisa_base_id, u.email_verificado,
       u.two_factor_enabled, u.whatsapp_phone, u.created_at,
       d.codigo AS divisa_base
FROM usuarios u
LEFT JOIN divisas d ON d.id_divisa = u.divisa_base_id
WHERE u.id_usuario = $1;  -- $1 = id_usuario del token
```

4.3 Listado de todos los usuarios C

Objetivo: listar cuentas ordenadas (pantalla de administración del emprendimiento).

Ruta: `GET /api/users` (`getAllUsers`)

```
SELECT u.id_usuario, u.nombre, u.email, u.divisa_base_id, u.email_verificado,
       u.created_at, d.codigo AS divisa_base
FROM usuarios u
LEFT JOIN divisas d ON d.id_divisa = u.divisa_base_id
ORDER BY u.id_usuario ASC;
```

4.4 Consulta por código de recuperación C

Objetivo: validar que el código de "recuperar contraseña" exista y **no haya expirado** (30 min).

Ruta: `POST /api/auth/reset-password`

```
SELECT * FROM usuarios
WHERE reset_token = '235873'
   AND reset_token_expires > CURRENT_TIMESTAMP
LIMIT 1;
```

4.5 Consulta por código de verificación de email C

```
SELECT * FROM usuarios
WHERE email = 'maria@example.com'
   AND verif_codigo = '482103'
   AND verif_expira > CURRENT_TIMESTAMP
LIMIT 1;
```

5. MODIFICACIÓN — Actualizar usuario (Update)

5.1 Cambio de perfil (nombre, email, moneda base) M

Objetivo: edición de datos personales.

Ruta: `PUT /api/users/me` (`updateUserProfile`)

```
UPDATE usuarios
SET nombre      = 'María García S.',
    email       = 'maria.g@example.com',
    divisa_base_id = 2,                -- 2 = EUR
    updated_at   = CURRENT_TIMESTAMP
WHERE id_usuario = 12
RETURNING id_usuario, nombre, email, divisa_base_id;
```

5.2 Cambio de contraseña M

Objetivo: actualizar el hash bcrypt y limpiar los códigos de recuperación usados.

Ruta: `PUT /api/users/me/password` y `POST /api/auth/reset-password`

```
UPDATE usuarios
SET password_hash      = '$2b$12$...nuevoHash...',
    reset_token         = NULL,
    reset_token_expires = NULL,
    updated_at          = CURRENT_TIMESTAMP
WHERE id_usuario = 12;
```

5.3 Confirmar email (código de 6 dígitos) M

```
UPDATE usuarios
SET email_verificado = TRUE,
    verific_codigo    = NULL,
    verific_expira    = NULL,
    verific_token     = NULL
WHERE id_usuario = 12;
```

5.4 Activar / desactivar doble factor (2FA) M

Ruta: `PUT /api/users/me/2fa`

```
UPDATE usuarios SET two_factor_enabled = TRUE, updated_at = CURRENT_TIMESTAMP
WHERE id_usuario = 12;
```

5.5 Guardar configuración de WhatsApp / alertas M

Ruta: `PUT /api/users/me/whatsapp` — semántica `COALESCE` (null no borra lo configurado).

```
UPDATE usuarios
SET whatsapp_phone = '+5491122334455',
```

```
whatsapp_api_key = 'apikey_callmebot...'
WHERE id_usuario = 12;
```

5.6 Guardar código de recuperación de contraseña M

Ruta: `POST /api/auth/forgot-password`

```
UPDATE usuarios
SET reset_token          = '235873',                -- código de 6 dígitos
    reset_token_expires = CURRENT_TIMESTAMP + INTERVAL '30 minutes'
WHERE email = 'maria@example.com';
```

6. BAJA — Eliminar usuario (Delete)

6.1 Baja definitiva D

Objetivo: eliminar la cuenta. Por las claves con **ON DELETE CASCADE** se borran automáticamente sus `favoritos`, `historial_de_consultas` y `alertas`.

Ruta: `DELETE /api/users/me` (`deleteUser`)

```
DELETE FROM usuarios
WHERE id_usuario = 12
RETURNING id_usuario;
```

6.2 Baja lógica (soft delete) D

Objetivo: marcar la cuenta como inactiva conservando el registro para auditoría (columna `activo` / `deleted_at` agregada en la migración 010). Las consultas de login deben filtrar `activo = TRUE`.

```
UPDATE usuarios
SET activo = FALSE, deleted_at = CURRENT_TIMESTAMP
WHERE id_usuario = 12;
```

Nota de diseño: Dolarito implementa la baja **definitiva** (el borrado de cuenta del usuario). La baja lógica queda demostrada a nivel de esquema y disponible para una futura gestión de administración.

7. Seguridad sobre las queries

- **Row Level Security (RLS):** cada usuario solo puede SELECT/UPDATE/DELETE su propio registro. La política usa el claim `id_usuario` del JWT (migración 006/010).
- **Nunca se guarda la contraseña:** solo su hash bcrypt (12 rondas).
- **Parámetros parametrizados:** toda consulta se ejecuta con sentencias preparadas (sin interpolación de strings en el backend).
- **Columna email UNIQUE:** evita duplicados; el alta con email repetido devuelve error y la UI lo muestra.

